```
class Lab04 {
    static void Main() {
        Console.WriteLine("Ready");
    }
}
```

Human Computer Interaction

Lab 04

Learning Objectives

By the end of this lab, students will be able to:

01

Use abstract classes and interfaces.

02

Store and query data with List, Dictionary, and LINQ.

03

Handle errors with try/catch and TryParse.

Abstract Classes

An abstract class cannot be instantiated. Its abstract methods have no body, and every derived class must override them.

Abstract methods end with a semicolon and have no body.

```
abstract class Shape
{
    public abstract double Area();
}

class Circle : Shape
{
    public double R;

    public override double Area()
    {
        return Math.PI * R * R;
    }
}
```

Interfaces

An interface lists members a class promises to provide. A class can implement many interfaces.

By convention, interface names start with I.

```
interface IPrintable
{
    void Print();
}

class Student : IPrintable
{
    public void Print()
    {
        Console.WriteLine("Student");
    }
}
```

Abstract vs interface

Aspect	abstract class / interface
Classes can use	Only one / many
Instance fields	Allowed / not allowed
Best for	Shared base code / shared capability

Problem 1:

Discount Interface

Create an interface IDiscountable and classes that implement it:

- Method double GetDiscountedPrice()
- Classes Book and Laptop implement it
- Each class uses its own discount rule

Then, in Main:

Store both kinds in a List<IDiscountable>.

Loop and print each discounted price.

Call the method through the interface type.

List<T>

List<T> is a resizable, strongly typed list: like a Python list that holds one type.

Add, Remove, Contains, Count, Sort

```
List<int> nums = new List<int>();
nums.Add(5);
nums.Add(2);
nums.Add(9);
nums.Remove(2);
nums.Sort();

foreach (int n in nums)
    Console.WriteLine(n);

Console.WriteLine(nums.Count);
```

Dictionary

A Dictionary<TKey, TValue> stores key-value pairs, like a Python dict. Keys must be unique.

Use TryGetValue when a key may be missing.

```
var grades = new Dictionary<string, int>();
grades["Ali"] = 85;
grades["Sara"] = 92;

if (grades.TryGetValue("Ali", out int g))
    Console.WriteLine(g);

foreach (var pair in grades)
    Console.WriteLine(pair.Key + ": " + pair.Value);
```

Problem 2:

Word Counter
Read a sentence from the user and:
Convert it to lower case with ToLower()
Split it into words with Split(' ')
Count each word using a Dictionary<string, int>
Print each word and its count
Print the word that appears most often

LINQ Queries

LINQ queries a collection with methods like Where, Select, and OrderBy. Think of filtering a DataFrame in Python.

s => s.Gpa >= 3.0 means: keep each s where Gpa is at least 3.0

```
using System.Linq;

var good = students
    .Where(s => s.Gpa >= 3.0)
    .OrderByDescending(s => s.Gpa)
    .ToList();

var names = students.Select(s => s.Name);
```

Where

Where keeps only the items that satisfy a condition. It returns a filtered sequence.

Usage: collection.Where(x => condition)

```
// Ali 3.5, Sara 4.0, Omar 2.5, Mona 3.0
var good = students.Where(s => s.Gpa >= 3.5);

foreach (var item in good)
    Console.WriteLine(item.Name);
// Ali, Sara
```

Select

Select transforms each item into something new, such as a single property. It returns a sequence of the results.

Usage: collection.Select(x => expression)

```
// Ali 3.5, Sara 4.0, Omar 2.5, Mona 3.0
var names = students.Select(s => s.Name);

foreach (string n in names)
    Console.WriteLine(n);
// Ali, Sara, Omar, Mona
```

OrderBy / OrderByDescending

OrderBy sorts from lowest to highest by a key. OrderByDescending sorts from highest to lowest. The original list is unchanged.

Usage: collection.OrderBy(x => key)

```
// Ali 3.5, Sara 4.0, Omar 2.5, Mona 3.0
var low = students.OrderBy(s => s.Gpa);
var high = students.OrderByDescending(s => s.Gpa);

// low:  Omar, Mona, Ali, Sara
// high: Sara, Ali, Mona, Omar
```

ToList

LINQ queries are lazy: they run when you use the result. ToList runs the query now and gives you a real List<T>.

Usage: query.ToList() (ToArray works the same for arrays)

```
// Ali 3.5, Sara 4.0, Omar 2.5, Mona 3.0
List<Student> good = students
    .Where(s => s.Gpa >= 3.5)
    .ToList();

Console.WriteLine(good.Count);  // 2
```

Count

Count returns how many items there are. With a condition, it counts only the matching items.

Usage: collection.Count(x => condition)

```
// Ali 3.5, Sara 4.0, Omar 2.5, Mona 3.0
int all = students.Count();
int high = students.Count(s => s.Gpa >= 3.5);

Console.WriteLine(all);   // 4
Console.WriteLine(high);  // 2
```

Sum

Sum adds up a numeric value from every item.

Usage: collection.Sum(x => number)

```
// Ali 3.5, Sara 4.0, Omar 2.5, Mona 3.0
double total = students.Sum(s => s.Gpa);

Console.WriteLine(total);  // 13
```

Average

Average returns the mean of a numeric value. It throws an exception if the list is empty.

Usage: collection.Average(x => number)

```
// Ali 3.5, Sara 4.0, Omar 2.5, Mona 3.0
double avg = students.Average(s => s.Gpa);

Console.WriteLine(avg);  // 3.25
```

Max / Min

Max and Min return the largest and smallest value. They return the value itself, not the item that has it.

Usage: collection.Max(x => number)

```
// Ali 3.5, Sara 4.0, Omar 2.5, Mona 3.0
double best = students.Max(s => s.Gpa);
double worst = students.Min(s => s.Gpa);

Console.WriteLine(best);   // 4
Console.WriteLine(worst);  // 2.5
```

FirstOrDefault

FirstOrDefault returns the first matching item, or null if nothing matches. Use it after OrderBy to get the top item.

Usage: collection.FirstOrDefault(x => condition)

```
// Ali 3.5, Sara 4.0, Omar 2.5, Mona 3.0
Student top = students
    .OrderByDescending(s => s.Gpa)
    .FirstOrDefault();

Student none = students
    .FirstOrDefault(s => s.Gpa > 4.0);

Console.WriteLine(top.Name);       // Sara
Console.WriteLine(none == null);   // True
```

Problem 3:

LINQ on Students
Create a List<Student> with at least 5 students (Name, Gpa, Year).
Using LINQ:
Print the **names** of students with Gpa above 3.0, sorted by Gpa.
Then print:

- The **average** Gpa of all students.
- The top student and the number of Year 2 students.

Bonus, using a Dictionary:

- Map each Year to its number of students.
- Print each year and its count.
- Use TryGetValue or ContainsKey.

Concepts review:

Q1. How many interfaces can a class implement?
A. Only one
B. Any number
C. None
D. Exactly two

Q2. What is true about an abstract class?
A. It can be created with new
B. It cannot be instantiated directly
C. It cannot contain any code
D. It must be static

Q3. What does a Dictionary store?
A. Duplicate keys in any order
B. Key-value pairs with unique keys
C. Only string values
D. Fixed-size arrays

Q4. What does Where(s => s.Gpa > 3) return?
A. A single student
B. The students whose Gpa is above 3
C. The average Gpa
D. A true/false value

Q5. Why use int.TryParse instead of int.Parse?
A. It parses larger numbers
B. It returns false instead of throwing on bad input
C. It only accepts positive numbers
D. It converts numbers to text

Exceptions: try/catch

An exception is a runtime error. Put risky code in try and handle it in catch. Catch specific exceptions first.

finally runs whether or not an exception occurred.

```
try
{
    int n = int.Parse(Console.ReadLine());
    Console.WriteLine(100 / n);
}
catch (FormatException)
{
    Console.WriteLine("Not a number.");
}
catch (DivideByZeroException)
{
    Console.WriteLine("Cannot divide by zero.");
}
finally
{
    Console.WriteLine("Done.");
}
```

int.TryParse

int.TryParse returns true or false instead of throwing, so it is the better choice for user input.

The parsed value comes back through an out parameter.

```
Console.Write("Enter age: ");
string text = Console.ReadLine();

if (int.TryParse(text, out int age))
{
    Console.WriteLine("Age: " + age);
}
else
{
    Console.WriteLine("Invalid input.");
}
```

Throwing Exceptions

Use throw to signal an error yourself, such as an invalid argument or a failed rule.

Catch it where you can recover from it.

```
public void Withdraw(double amount)
{
    if (amount > Balance)
        throw new Exception("Not enough funds");
    Balance -= amount;
}

try { acc.Withdraw(500); }
catch (Exception ex)
{
    Console.WriteLine(ex.Message);
}
```

Problem 4:

Write a console program that:
Reads a name and Gpa in a loop until the user types done.
Validates the Gpa with double.TryParse (0 to 4).
Stores valid students in a List<Student>.
Then, using LINQ, prints:

- The students sorted by Gpa
- The average Gpa
- The top student
- The number of students above 3.0

Bonus: show an error message instead of crashing on any bad input.